

Practice Set: Structures in C++

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define a **Structure** in C++. Explain the four primary limitations of structures in traditional C programming (variables only, no data hiding, non-first-class types, and mandatory tag requirement).

Question 2: How does C++ expand the capabilities of structures to align with Object-Oriented Programming (OOP)? Detail how C++ structures support the principle of Encapsulation.

Question 3: Explain the difference in physical RAM memory allocation between a structure type blueprint/definition and an instantiated structure variable.

Question 4: Map out the memory layout of the following structure in RAM when instantiated as book b1; starting at memory address 0x7ffe00:

```
struct book
{
    int bookid;    // 4 Bytes
    char title[20]; // 20 Bytes
    float price;   // 4 Bytes
};
```

Calculate the total memory allocated and the specific address offsets for each member variable.

Question 5: Compare **Structures in Standard C** versus **Structures in Modern C++** across five criteria: Allowed Members, Encapsulation, Access Specifiers, Variable Instantiation Syntax, and Member Access Mechanics.

Question 6: Compare a **struct** and a **class** in C++. What are the two fundamental differences between them regarding default member access visibility and default inheritance visibility?

Question 7: Contrast the procedural C-style approach of manipulating structure data using external standalone functions versus the modern C++ object-oriented approach of using internal member functions.

Question 8: Explain why member functions defined inside a C++ structure can access member variables directly without using the dot (.) member access operator.

Question 9: Discuss the legacy console functions `clrscr()` and `getch()` from `<conio.h>`. Why are they deprecated, and why should they be avoided in modern standard C++ compilers (GCC, Clang, MSVC)?

Question 10: Write a complete C++ program that defines an encapsulated structure `Student` with data members `rollNo` (int), `name` (char[30]), and `marks` (float), and internal member functions `input()` and `display()`. Instantiate a student instance in `main()` and invoke its member functions.

Section 2: Interview & Viva Questions (11-20)

Question 11 (Viva): How many bytes of RAM are consumed when a struct definition blueprint is loaded by the compiler before any variable is instantiated?

Question 12 (Viva): Is the `struct` keyword mandatory when declaring a structure variable instance in C++ as it was in standard C?

Question 13 (Viva): What is the default access specifier for member variables and member functions declared inside a C++ struct?

Question 14 (Viva): What is the default access specifier for members of a C++ class?

Question 15 (Viva): When a derived struct inherits from a base struct without specifying an inheritance access specifier, what mode of inheritance is used by default?

Question 16 (Viva): Can a C++ structure contain constructors, destructors, and access specifiers like `private` and `protected`?

Question 17 (Viva): Under what software architectural scenario is it best practice to choose a struct over a class in modern C++?

Question 18 (Viva): Under what software architectural scenario is it best practice to choose a class over a struct in modern C++?

Question 19 (Viva): Why was the non-standard header `<conio.h>` commonly used in legacy environments like Turbo C++?

Question 20 (Viva): How does internal variable access inside a struct member function differ syntactically from accessing struct members in an external non-member function?

Answers & Explanations

Answer: 01

Definition: A Structure (defined via the struct keyword) is a user-defined compound data type that groups variables of different data types under a single cohesive unit name.

Limitations in Standard C:

Variables Only: Can only hold data variables; cannot contain member functions.

No Data Hiding: Lacks access specifiers (public, private, protected); all members are unconditionally public and globally accessible.

Not Full Built-in Types: Standard operators (like + or -) cannot operate directly on user-defined structs.

Mandatory struct Tag: Declaring an instance requires repeating the struct keyword (e.g., struct book b1;) unless explicitly aliased with typedef.

Answer: 02

C++ OOP Expansions:

Member Functions: Structures can contain both data members (variables) and member functions (methods).

Encapsulation: By packaging state variables and manipulating functions together into a single unit, C++ structures satisfy the OOP principle of Encapsulation.

Direct Variable Access: Member functions have direct, privileged access to internal variables without requiring the dot (.) operator.

Type Name Promotion: The structure tag becomes a first-class user-defined type name, making `book b1;` fully valid syntax without the `struct` keyword.

Answer: 03

Structure Blueprint/Definition: Allocates **0 bytes** of physical RAM at runtime. It acts purely as a compiler schema dictating how to layout memory later.

Structure Variable (Instance): Allocates a contiguous block of physical RAM memory on the stack (or heap) to store member variables sequentially based on their data types and compiler alignment padding.

Answer: 04

Total Memory Allocated: $4 + 20 + 4 = 28$ Bytes\$.

Address Offsets:

bookid (4 Bytes): 0x7ffe00 to 0x7ffe03

title (20 Bytes): 0x7ffe04 to 0x7ffe17

price (4 Bytes): 0x7ffe18 to 0x7ffe1b

Next available memory address: 0x7ffe1c

Answer: 05

Feature	Structures in Standard C	Structures in Modern C++
Allowed Members	Data variables only.	Both data variables and member functions.
Encapsulation	Not supported (passive data container).	Fully supported (bundles state + behavior).
Access Specifiers	None (all members are public).	Supported (public, private, protected).
Variable Instantiation	Requires struct tag (struct book b1;).	Direct type name (book b1;).

Feature	Structures in Standard C	Structures in Modern C++
Member Access	External functions use <code>b1.member</code> .	Member functions access variables directly.

Answer: 06

In C++, struct and class are nearly identical; both support member functions, constructors, inheritance, and polymorphism.

Default Differences:

Default Member Access: Members of a struct are **public** by default, whereas members of a class are **private** by default.

Default Inheritance Visibility: Struct inheritance defaults to **public**, whereas class inheritance defaults to **private**.

Answer: 07

C-Style Procedural Approach: Requires defining external, non-member functions (e.g., `book inputBookData()`) that pass struct objects by value or pointer, accessing individual fields through instances using dot notation (`b.bookid`).

C++ OOP Approach: Member functions (e.g., `b1.input()`) are placed directly inside the structure definition. These functions execute within the scope of the instance and directly read/write internal member fields without explicit dot notation.

Answer: 08

Because member functions execute within the class/struct scope, the compiler automatically binds unqualified member variable names to the instance invoking the method, giving them direct privileged access without requiring the dot (`.`) operator.

Answer: 09

`clrscr()` was used to clear the MS-DOS screen, and `getch()` was used to pause console execution by awaiting a key press.

Both belong to the obsolete `<conio.h>` header from 16-bit MS-DOS environments (Turbo C++). They are non-standard artifacts unsupported by standard C++ compilers (GCC, Clang, MSVC). Modern IDEs preserve console outputs automatically.

Answer: 10

```
#include <iostream>
using namespace std;
struct Student
{
    int rollNo;
    char name[30];
    float marks;

    // Member function to input data
    void input() {
        cout << "Enter Roll No, Name, and Marks: ";
        cin >> rollNo >> name >> marks;
    }

    // Member function to display data
    void display() {
        cout << "Roll No: " << rollNo
            << " | Name: " << name
            << " | Marks: " << marks << endl;
    }
};

int main() {
    Student s1; // 'struct' keyword is optional in C++
    s1.input();

    cout << "\nDisplaying Student Record:" << endl;
    s1.display();

    return 0;
}
```

Answer: 11

0 bytes; the struct definition is purely a type schema for the compiler.

Answer: 12

No; in C++, the structure tag is promoted to a first-class type name, making the struct keyword optional during instantiation.

Answer: 13

public.

Answer: 14

private.

Answer: 15

public inheritance.

Answer: 16

Yes; C++ structures support full OOP capabilities including constructors, destructors, and private/protected access specifiers.

Answer: 17

For plain-old-data (POD) containers where all data fields are inherently public and do not require encapsulation or invariants (e.g., 2D coordinates {int x, y;} or RGB color triplets).

Answer: 18

Whenever data hiding is needed to protect internal state with private members and validate invariants through public interface methods.

Answer: 19

Primarily for `clrscr()` to clear the terminal and `getch()` to pause execution so the console window did not close immediately upon termination.

Answer: 20

Internal member functions access variables directly by name without the dot operator, whereas external non-member functions must use the instance identifier paired with the dot operator (e.g., `b1.bookid`).